WISE ROUND 3: SYSTEM DESIGN — UPDATED GUIDE

VIPER Architecture | Spend Squad | Descriptive + Code

60 min | Two Engineers | Zoom

WHAT WISE OFFICIALLY ASSESSES (From their website)

4 criteria — in order of importance for a Manager candidate:

1. **Product thinking:** "Think from the customer's perspective and understand the product needs." Start with user journey, not database schema.
2. **Problem solving:** "Approach to solving a non-trivial problem within a set time." Structured breakdown, don't jump to solution.
3. **Communication:** "Justify your decisions. We want to understand WHY you approach problems a certain way." Every decision needs "because..."
4. **Technical proficiency:** "Knowledge relevant to Wise's technical stack." VIPER, TCA, SwiftUI, Combine, Kafka, Dynamic Forms.

THE ROUND STRUCTURE

0:00 - 0:05	Introductions
0:05 - 0:15	Experience discussion (your past projects, bugs, decisions)
0:15 - 0:20	They present the design problem
0:20 - 0:25	You clarify requirements
0:25 - 0:40	You design the architecture
0:40 - 0:50	They add requirements / probe deeper
0:50 - 0:55	Trade-offs, testing, production
0:55 - 1:00	Your questions

PART 1: EXPERIENCE DISCUSSION (First 10 min)

From Wise's page: "*Expect questions about your past experiences and projects. Your contributions, decisions you have taken, recent bugs you've fixed.*"

Story 1: "Tell me about a technical decision you made"

I led the BFX project at PayPal — re-enabling currency conversion for Visa multi-currency cards. This was a $15M+ revenue opportunity but with regulatory risk since Visa had previously mandated we stop offering this

service.

The key decision was rollout strategy. Product wanted to launch across all countries simultaneously for faster revenue. I advocated for a phased country-by-country approach — Italy and Saudi Arabia first, then France and South Africa, then Lithuania.

I built validation checklists for each country: verify BINs in the FRED system, test in staging, deploy config changes, run the batch job, and validate post-deployment. I also created monitoring dashboards tracking conversion rates and error rates per country.

We caught a BIN configuration issue in Italy during the first ramp. Certain card types weren't processing correctly. If we'd launched all countries simultaneously, this would have caused failed conversions for customers across multiple markets. The phased approach let us fix Italy before expanding.

The result: we completed the rollout on schedule, approximately 45% of users chose PayPal conversion when offered, and we avoided a multi-country incident.

What to SAY:

> "The decision was to trade launch speed for customer safety. In fintech, that's always the right trade-off. This maps directly to how I'd approach feature rollouts at Wise — behind feature flags, country by country, with monitoring at each stage."

Story 2: "Tell me about a bug you recently fixed"

On the RUNE project at PayPal, QA reported that after logging into the iOS app, the Send/Request page displayed abnormally — buttons appeared that shouldn't be visible, and the page title was missing. The bug only occurred on fresh login. Killing and relaunching the app showed the correct UI.

I traced the issue to the experiment flag `Trmt_p2p_money_movement_flow`. The `isExperimentEnabled` function was returning nil for `qualifiedTreatment` during fresh login. On app relaunch, the treatment was correctly available because it had been cached from the previous session.

The root cause was that experiment assignments were fetched asynchronously after login, but the UI rendered before the fetch completed. Cached values from the previous session were cleared on logout, so fresh login had no cached values to fall back on.

I documented the root cause with exact build versions and reproduction steps, then recommended the fix: load cached experiment values synchronously on startup, refresh from the server in the background. This two-phase approach ensures the UI always has flag values available, even on fresh login.

What to SAY:

> "This is a common pattern in mobile apps — the timing between async initialization and UI rendering. It applies to feature flags, experiment assignments, remote config, and even Dynamic Forms. The general solution is always the same: serve cached values first, refresh in background."

Story 3: "Tell me about a system you designed"

I created the complete BFX knowledge transfer documentation at PayPal. The project had critical knowledge concentrated in one engineer — the original implementer. When we needed to expand, new team members had

no way to understand the system without extensive shadowing. This was a bus factor risk.

I built a comprehensive Confluence reference: project context and regulatory background, system architecture diagrams, step-by-step deployment process with screenshots, pre-ramp and post-ramp validation checklists, troubleshooting guide for the 10 most common issues, access request templates, and a 4-week onboarding plan progressing from shadow to supervised to independent.

The next engineer onboarded in 1 week instead of 4 and executed their first ramp independently by week 3. The document became the team's living reference with a quarterly review cycle.

What to SAY:

> "As a manager, my job is to make myself replaceable on execution while focusing on strategic decisions. Documentation that enables the team to operate independently is one of the highest-leverage things a lead can do."

PART 2: THE DESIGN FRAMEWORK

Step 1: Clarify (2-3 min)

When they present the problem, always ask these:

User-focused (shows product thinking):

> "What's the core user journey? What does the customer see step by step?"
> "What's the latency expectation? Should this feel instant?"
> "Should this work offline? What's the degraded experience?"

Technical (shows engineering depth):

> "Is there an existing backend API, or am I designing that too?"
> "Are there compliance or regulatory constraints for this feature?"
> "Should I focus on iOS architecture, or include the backend flow?"

Scope (shows prioritization):

> "Should I start with the happy path and discuss error handling after?"
> "Should I consider both platforms, or focus on iOS?"

Step 2: Start from the Customer (2 min)

ALWAYS start with the user journey, NOT the architecture.

> "Let me start with what the customer experiences:
> Step 1: [First thing user sees/does]
> Step 2: [Next action]
> Step 3: [Outcome]
> Now let me design the architecture that makes this happen..."

This is the #1 thing that separates passing candidates from failing ones at Wise. Their assessment explicitly says "product thinking" — starting from the customer proves it.

Step 3: Draw High-Level Architecture (3 min)

Draw three layers with clear arrows:

```
┌──────────────────────────────────────────────────────────────┐
│ EXTERNAL SYSTEMS                                             │
│ Visa/Mastercard | APNs | Merchant | Banking Partners         │
└──────────────────────────────────────────────────────────────┘
                    │
┌──────────────────────────────────────────────────────────────┐
│ WISE BACKEND                                                 │
│ API Gateway → Services → Event Bus (Kafka)                   │
│                    │                │                        │
│          ┌─────────┴────────┐       │                        │
│          ▼                  ▼       │                        │
│      Notification    Transaction/Card                        │
│      Service         Service                                 │
│      (→ APNs)        (→ PostgreSQL)                          │
└──────────────────────────────────────────────────────────────┘
                    │
┌──────────────────────────────────────────────────────────────┐
│ MOBILE APP (VIPER)                                           │
│ Push Handler → Router → Presenter → Interactor               │
│                              → Repository                    │
│                                ↙     ↘                       │
│                            APIClient   Cache                 │
└──────────────────────────────────────────────────────────────┘
```

What to SAY:

> "Three layers — external systems Wise integrates with, Wise's backend services connected by Kafka for event-driven communication, and the mobile app using VIPER architecture. I know Wise uses VIPER and TCA — I'd use VIPER for this feature since it's what the existing codebase uses and I'm experienced with it. Let me zoom into the mobile architecture..."

Step 4: Draw VIPER Module (5 min)

```
┌──────────────────────────────────────────────────────────────┐
│ BUILDER (assembles the module)                               │
│                                                              │
│ VIEW (Protocol)                                              │
│ │ Shows UI. Completely dumb — zero logic.                    │
│ │ Sends user actions to Presenter.                           │
│ │                                                            │
```

```
│ ↕                                   │
│ PRESENTER                           │
│ │ Receives user actions from View.  │
│ │ Asks Interactor for data.         │
│ │ Formats data for View (Decimal → "S$65.25").  │
│ │ Tells Router to navigate.         │
│ │                                   │
│ ↕                 ↕                 │
│ INTERACTOR          ROUTER          │
│ │ Business logic.   │ Navigation.   │
│ │ Fetches/validates data. │ Pushes screens.  │
│ │ Coordinates repos.  │ Presents modals.  │
│ │                   │ Deep link handling. │
│ ↕                                   │
│ ENTITY                              │
│ │ Pure data models.                 │
│ │ No dependencies. No frameworks.   │
│                                     │
│ REPOSITORY (called by Interactor)   │
│ ├── APIClient (network — URLSession)  │
│ └── LocalCache (storage — CoreData/Keychain)  │
└──────────────────────────────────────────────────┘
```

The 5 things to say about each component:

View: "The View is completely dumb — it only renders what the Presenter tells it. No business logic, no data formatting, no network calls. This means I can swap the View from UIKit to SwiftUI without touching any other component."

Presenter: "The Presenter is the coordinator. It receives user actions from the View, asks the Interactor for data, formats the results for display, and tells the Router to navigate. It's the only component that talks to both View and Interactor."

Interactor: "Pure business logic — like a Use Case in Clean Architecture. It validates input, coordinates data fetching through the Repository, and applies business rules. It doesn't know about any UI framework."

Router: "Owns all navigation. When the user taps a transaction, the Presenter tells the Router, and the Router pushes the detail screen. Navigation logic is isolated — changing how screens connect doesn't touch business logic."

Entity: "Pure data models. No dependencies, no frameworks. Just the data structures — Transaction, Quote, SpendControls. These are shared across the entire feature."

Step 5: Deep Dive on the Hard Part (15 min)

Every feature has ONE hard part. Identify it and go deep:

Feature	The Hard Part	What to Discuss
Card transactions	Real-time push + multi-currency	Notification flow, optimistic UI, FX display
Money transfer	Rate quoting + idempotency	Debounce, TTL, duplicate prevention
3DS authentication	Time-limited approval	5-min expiry, biometric, deep linking
Multi-currency balance	Cached vs fresh data	Optimistic UI, encrypted storage
Exchange rates	Real-time updates	Polling vs WebSocket trade-off
Event-driven data	Multiple event types	Event router pattern, data buckets

Step 6: When They Add Requirements (10 min)

They'll say: "*Now what if we add X?*"

Template response:

> "I'd create a new VIPER module for this rather than bloating the existing one — Single Responsibility. I'd wrap it behind a feature flag for gradual rollout. The Interactor handles the new business logic, the Presenter formats for display, and the Router handles any new navigation.
> The trade-off is [A vs B]. I'd choose [A] because [reason]. If [requirement changes], I'd reconsider."

Step 7: Close with Production Concerns (5 min)

Always volunteer these without being asked:

Testing:

> "Each VIPER component is independently testable. Mock the View to test Presenter logic. Mock the Repository to test Interactor business rules. No real network, no real UI — tests run in milliseconds. For a fintech app processing £26 billion, this isolation is essential."

Monitoring:

> "I'd track: push notification delivery latency under 1 second, transaction list load time under 300ms, API error rate per endpoint, crash-free rate above 99.9%, and funnel conversion for multi-step flows."

Feature flags:

> "Every new feature behind a flag. Start with 5% of users, monitor for a week, ramp to 25%, 50%, 100%. Kill switch for emergency disable. This is exactly how I manage BFX rollouts at PayPal — country by country with monitoring dashboards."

Security:

> "Certificate pinning on all API calls. Sensitive data in Keychain, not UserDefaults. Biometric auth for high-value actions. Screen protection — blur content when app enters background so card details aren't visible in

> the app switcher."

PART 3: TRANSFER FEATURE (VIPER VERSION)

Their Prompt:

> "Wise has a transfer feature where users send money internationally. Walk me through how you'd expect the mobile architecture to work."

User Journey (start here):

> "The customer opens Wise, enters an amount, sees the real-time rate and fee, reviews the quote, confirms with biometric auth, and tracks the transfer status until delivery. Let me design the architecture that makes this happen."

VIPER Modules:

```
TransferFlowCoordinator
│
├── TransferAmountModule (VIPER)
│   ├── View: Amount input + currency selector + live quote display
│   ├── Presenter: Formats rate, fee, converted amount for display
│   ├── Interactor: Debounce → fetch quote → validate → cache with TTL
│   ├── Router: Delegates to Coordinator for next step
│   └── Entity: TransferQuote, CurrencyPair
│
├── RecipientModule (VIPER)
│   ├── View: Recipient list / search / add new
│   ├── Presenter: Format recipient display, validate selection
│   ├── Interactor: Fetch recipients, Dynamic Forms for new recipient
│   ├── Router: Delegates to Coordinator
│   └── Entity: Recipient (varies by country — IBAN, sort code, IFSC)
│
├── ReviewModule (VIPER)
│   ├── View: Summary — amount, fee, rate, recipient, delivery estimate
│   ├── Presenter: Format all details, show quote countdown
│   ├── Interactor: Validate quote not expired, biometric auth, submit
│   ├── Router: Navigate to tracking
│   └── Entity: TransferConfirmation
│
└── TrackingModule (VIPER)
    ├── View: Status timeline + estimated delivery
    ├── Presenter: Format status steps
    ├── Interactor: Poll for updates + handle push notifications
```

```
├── Router: Deep link from push → specific transfer
└── Entity: TransferStatus
```

Code — The Critical Parts:

Rate Fetching with Debounce:

```swift
// ═══ INTERACTOR — Rate Fetching ═══

class TransferAmountInteractor: TransferAmountInteractorProtocol {
    weak var output: TransferAmountInteractorOutput?
    private let rateRepository: RateRepositoryProtocol
    private var debounceTask: Task<Void, Never>?

    func onAmountChanged(_ amount: String) {
        // Cancel previous debounce
        debounceTask?.cancel()

        guard let decimal = Decimal(string: amount), decimal > 0 else {
            output?.didClearQuote()
            return
        }

        // Wait 500ms after user stops typing
        debounceTask = Task {
            try? await Task.sleep(nanoseconds: 500_000_000)
            guard !Task.isCancelled else { return }
            await fetchQuote(amount: decimal)
        }
    }

    private func fetchQuote(amount: Decimal) async {
        output?.didStartLoading()
        do {
            let quote = try await rateRepository.getQuote(
                amount: amount, from: "USD", to: "EUR"
            )
            await MainActor.run { output?.didFetchQuote(quote) }
        } catch {
            await MainActor.run { output?.didFailQuote(error) }
        }
    }
}
```

What to SAY:

> "I debounce the amount input by 500 milliseconds. When the user types '1000', that's four keystrokes — without debouncing, we'd fire four API calls. With debounce, we wait until they stop typing, then fire one call. This saves bandwidth, reduces server load, and prevents UI flickering.
> I use Task-based debounce with cancellation — if the user types another character within 500ms, the old task is cancelled and a new one starts. This is cooperative cancellation from Swift's structured concurrency."

Repository — Cache vs Network:

```swift
// ═══ REPOSITORY — Cache-First Strategy ═══

class RateRepository: RateRepositoryProtocol {
    private let apiClient: TransferAPIClientProtocol
    private let cache: RateCacheProtocol
    private let ttl: TimeInterval = 30  // 30 seconds — matches Wise's actual TTL

    func getQuote(amount: Decimal, from: String, to: String) async throws -> TransferQuote {
        let cacheKey = "\(from)-\(to)"

        // Check cache first
        if let cached = cache.get(cacheKey), !cached.isExpired(ttl: ttl) {
            // Recalculate with cached rate but new amount
            return TransferQuote(
                sourceAmount: amount,
                rate: cached.rate,
                fee: calculateFee(amount: amount),
                targetAmount: (amount - calculateFee(amount: amount)) * cached.rate,
                expiresAt: cached.fetchedAt.addingTimeInterval(ttl)
            )
        }

        // Cache miss — fetch from API
        let quote = try await apiClient.fetchQuote(amount: amount, from: from, to: to)
        cache.set(cacheKey, rate: quote.rate, fetchedAt: Date())
        return quote
    }

    private func calculateFee(amount: Decimal) -> Decimal {
        max(amount * Decimal(string: "0.005")!, Decimal(string: "1.00")!)
    }
}
```

What to SAY:

> "Cache-first strategy. Show the user a quote instantly from cached rate, then refresh in the background if the cache is near expiry. The TTL is 30 seconds — matching Wise's actual rate guarantee window. If the cache has expired, we fetch fresh from the API.
> This is the same pattern as my Round 2 solution — the LRU Cache with TTL. The concepts transfer directly."

Transfer Confirmation — Idempotency:

```swift
```

```
// ═══ INTERACTOR — Confirmation ═══

class TransferConfirmInteractor: TransferConfirmInteractorProtocol {
    weak var output: TransferConfirmInteractorOutput?
    private let apiClient: TransferAPIClientProtocol
    private let biometricAuth: BiometricAuthProtocol
    private let idempotencyStore: IdempotencyStoreProtocol

    func confirmTransfer(quote: TransferQuote, recipientId: String) {
        Task {
            // Step 1: Verify quote hasn't expired (client-side check)
            guard !quote.isExpired else {
                await MainActor.run { output?.didExpireQuote() }
                return
            }

            // Step 2: Biometric authentication
            guard try await biometricAuth.authenticate(
                reason: "Confirm transfer of \(quote.sourceAmount) \(quote.sourceCurrency)"
            ) else {
                await MainActor.run { output?.didCancelAuth() }
                return
            }

            // Step 3: Generate idempotency key
            let idempotencyKey = idempotencyStore.getOrCreate(
                for: "\(quote.id)-\(recipientId)"
            )

            // Step 4: Submit transfer
            do {
                let transfer = try await apiClient.createTransfer(
                    quote: quote,
                    recipientId: recipientId,
                    idempotencyKey: idempotencyKey
                )
                await MainActor.run { output?.didCreateTransfer(transfer) }
            } catch {
                await MainActor.run { output?.didFailTransfer(error) }
            }
        }
    }
}
```

What to SAY:

> "The confirmation flow has four critical steps. First, client-side quote expiry check — don't even attempt if expired. Second, biometric authentication — Face ID or Touch ID before any money moves. Third, idempotency key generation — if the network drops after confirmation, the user might tap again. Without idempotency, we'd create two transfers. With it, the backend recognizes the duplicate and returns the same transfer. Fourth, the actual API call.
> Idempotency is non-negotiable in fintech. This is the same pattern as my Round 2 pair programming solution where the token manager ensured concurrent calls didn't trigger multiple refreshes."

Status Tracking:

```swift
// ═══ INTERACTOR — Status Polling ═══

class TransferTrackingInteractor: TransferTrackingInteractorProtocol {
    weak var output: TransferTrackingInteractorOutput?
    private let apiClient: TransferAPIClientProtocol
    private var pollingTask: Task<Void, Never>?

    func startTracking(transferId: String) {
        pollingTask = Task {
            while !Task.isCancelled {
                do {
                    let status = try await apiClient.getTransferStatus(transferId)
                    await MainActor.run { output?.didUpdateStatus(status) }

                    // Stop polling when terminal state reached
                    if status.isTerminal { break }
                } catch {
                    // Silent retry — tracking is background activity
                }
                // Poll every 5 seconds
                try? await Task.sleep(nanoseconds: 5_000_000_000)
            }
        }
    }

    func stopTracking() {
        pollingTask?.cancel()
    }
}
```

What to SAY on the Poll vs WebSocket trade-off:

> "I'd start with polling every 5 seconds. WebSocket is more real-time but adds complexity — connection management, reconnection logic, and background/foreground handling on iOS. For transfer status that changes on a minutes-not-milliseconds timescale, polling is simpler and sufficient.

> The trade-off: polling wastes some bandwidth with 'nothing changed' responses, but it's simple, stateless, and works reliably across network transitions. If we later need real-time — like a live rate ticker — WebSocket makes more sense.
> I use Task-based polling with cancellation — when the user leaves the screen, the View disappears, the Interactor's `stopTracking()` is called, and the polling Task is cancelled. No orphaned background work."

PART 4: MULTI-CURRENCY BALANCE (VIPER VERSION)

Their Prompt:

> "Wise shows users their balances in multiple currencies. How would you architect this on mobile?"

User Journey:

> "The customer opens the app and immediately sees all their currency balances — SGD, EUR, GBP. They can tap any currency to see transactions. They can pull to refresh for the latest balance. The total is shown in their base currency."

Code:

```swift
```

```swift
// ═══ ENTITY ═══

struct WalletBalance: Codable, Identifiable {
    let id: String
    let currency: String
    let amount: Decimal
    let lastUpdated: Date
}

struct WalletState {
    let balances: [WalletBalance]
    let totalInBaseCurrency: Decimal
    let baseCurrency: String
}

// ═══ INTERACTOR ═══

class BalanceInteractor: BalanceInteractorProtocol {
    weak var output: BalanceInteractorOutput?
    private let repository: BalanceRepositoryProtocol

    func fetchBalances() {
        // Phase 1: Show cached immediately (fast UI)
        if let cached = repository.getCachedBalances() {
            output?.didFetchBalances(cached, source: .cache)
        }

        // Phase 2: Refresh from network in background
        Task {
            do {
                let fresh = try await repository.fetchFreshBalances()
                await MainActor.run {
                    output?.didFetchBalances(fresh, source: .network)
                }
            } catch {
                await MainActor.run {
                    // Only show error if we have no cached data
                    if repository.getCachedBalances() == nil {
                        output?.didFailFetchBalances(error)
                    }
                    // If we have cached data, silently fail — user sees stale but valid data
                }
            }
        }
    }
}
```

```
// ═══ PRESENTER ═══

class BalancePresenter: BalancePresenterProtocol {
    weak var view: BalanceViewProtocol?
    var interactor: BalanceInteractorProtocol
    var router: BalanceRouterProtocol

    private let formatter = CurrencyFormatter()
}

extension BalancePresenter: BalanceInteractorOutput {
    func didFetchBalances(_ state: WalletState, source: DataSource) {
        let viewModels = state.balances.map { balance in
            BalanceCellViewModel(
                currency: balance.currency,
                flag: flagEmoji(for: balance.currency),
                formattedAmount: formatter.format(balance.amount, currency: balance.currency),
                lastUpdated: source == .cache ? "Cached" : "Just now"
            )
        }

        let totalFormatted = formatter.format(
            state.totalInBaseCurrency, currency: state.baseCurrency
        )

        view?.showBalances(viewModels, total: totalFormatted)

        if source == .cache {
            view?.showStaleIndicator("Updating...")
        } else {
            view?.hideStaleIndicator()
        }
    }
}

enum DataSource { case cache, network }
```

What to SAY on the trade-off — Optimistic UI vs Wait for Fresh:

> "I show cached balances immediately because waiting for a network call on every app open creates perceived slowness. The risk is showing a stale balance — but I mitigate this with a 'Updating...' indicator and the 'Last updated' timestamp. If a user just completed a transfer, I optimistically deduct the amount locally while the backend confirms.
> The key UX decision: cached balance with a stale indicator is better than a loading spinner for 2 seconds. Users open the app to check their balance — they want to see it NOW, even if it's a few minutes old."

PART 5: EXCHANGE RATE DISPLAY

Their Prompt:

> "We want to show live exchange rates that update regularly. How would you design this?"

The Trade-off Discussion (This is where the Reddit candidate failed):

Approach	How it Works	Pros	Cons	When to Use
Polling	Timer fires every N seconds, makes API call	Simple, stateless, works with REST	Wastes bandwidth when rate unchanged, battery drain	Most screens — transfer form, balance
WebSocket	Persistent connection, server pushes updates	True real-time, efficient (only sends changes)	Connection management, reconnection logic, background/foreground handling	Rate ticker screen, live charts
SSE	Server-Sent Events, one-way stream	Simpler than WebSocket, works with HTTP	One-directional only (server → client)	Rate feed with no client interaction

What to SAY:

> "I'd use polling for most screens and WebSocket for the rate ticker. Here's why: the transfer form needs a rate when the user enters an amount — a 30-second poll interval is fine because the rate is only 'locked' when they get a quote. But a rate ticker showing live rates needs sub-second updates — polling every second would drain the battery and waste bandwidth. WebSocket lets the server push only when the rate actually changes.
> The trade-off: WebSocket adds complexity — I need to handle connection drops, reconnect with exponential backoff, disconnect when the app backgrounds, and reconnect on foreground. For a single rate ticker screen, that's justified. For every screen in the app, it's overkill."

Mobile-Specific Concerns:

```swift
```

```swift
// ═══ WebSocket Manager with Lifecycle Awareness ═══

class RateWebSocketManager {
    private var webSocketTask: URLSessionWebSocketTask?
    private let url: URL

    func connect() {
        webSocketTask = URLSession.shared.webSocketTask(with: url)
        webSocketTask?.resume()
        listenForUpdates()
    }

    func disconnect() {
        webSocketTask?.cancel(with: .goingAway, reason: nil)
    }

    private func listenForUpdates() {
        webSocketTask?.receive { [weak self] result in
            switch result {
            case .success(let message):
                self?.handleMessage(message)
                self?.listenForUpdates()  // Continue listening
            case .failure:
                self?.reconnectWithBackoff()
            }
        }
    }

    // App lifecycle handling
    func onAppDidEnterBackground() {
        disconnect()  // Save battery — no WebSocket in background
    }

    func onAppWillEnterForeground() {
        connect()  // Reconnect when user returns
    }
}
```

What to SAY:

> "Battery management is critical. I disconnect the WebSocket when the app backgrounds and reconnect on foreground. iOS will terminate background connections anyway after a few minutes, so it's better to disconnect cleanly and reconnect deterministically."

PART 6: EVENT-DRIVEN ARCHITECTURE (Confirmed Glassdoor Mar 2025)

Their Prompt:

> "Design for event-driven architecture and data buckets"

This is about how the mobile app receives and processes different types of backend events.

Architecture:

```swift
```

```
// ═══ Event Router — routes events to the right handler ═══

protocol EventHandler {
    var eventType: String { get }
    func handle(_ data: Data) throws
}

class EventRouter {
    private var handlers: [String: EventHandler] = [:]

    func register(_ handler: EventHandler) {
        handlers[handler.eventType] = handler
    }

    func dispatch(eventType: String, data: Data) {
        guard let handler = handlers[eventType] else {
            // Unknown event type — log but don't crash
            print("Unknown event: \(eventType)")
            return
        }
        try? handler.handle(data)
    }
}

// ═══ Specific Handlers ═══

class TransferStatusHandler: EventHandler {
    let eventType = "transfer.status.changed"
    private let transferStore: TransferStoreProtocol

    func handle(_ data: Data) throws {
        let update = try JSONDecoder().decode(TransferStatusUpdate.self, from: data)
        transferStore.updateStatus(transferId: update.id, status: update.newStatus)
    }
}

class BalanceChangeHandler: EventHandler {
    let eventType = "balance.changed"
    private let balanceStore: BalanceStoreProtocol

    func handle(_ data: Data) throws {
        let update = try JSONDecoder().decode(BalanceUpdate.self, from: data)
        balanceStore.updateBalance(currency: update.currency, amount: update.newAmount)
    }
}
```

```swift
class CardTransactionHandler: EventHandler {
    let eventType = "card.transaction.created"
    private let transactionStore: TransactionStoreProtocol

    func handle(_ data: Data) throws {
        let txn = try JSONDecoder().decode(CardTransaction.self, from: data)
        transactionStore.insert(txn)
    }
}

// ═══ Registration ═══

let router = EventRouter()
router.register(TransferStatusHandler(transferStore: transferStore))
router.register(BalanceChangeHandler(balanceStore: balanceStore))
router.register(CardTransactionHandler(transactionStore: transactionStore))

// Events arrive via push notification or WebSocket:
// router.dispatch(eventType: "card.transaction.created", data: pushData)
```

What to SAY about "Data Buckets":

> "Data buckets means partitioning state by domain. Each bucket has its own cache policy, refresh strategy, and TTL:
> Transfer bucket: transfer history and status. TTL: 5 minutes. Refresh: poll during active transfer, otherwise on-demand.
> Balance bucket: currency balances. TTL: 2 minutes. Refresh: on app foreground + after any transaction.
> Rate bucket: exchange rates. TTL: 30 seconds. Refresh: debounced on amount input.
> Card bucket: transactions and card details. TTL: 5 minutes. Refresh: on push notification + pull-to-refresh.
> Each bucket is independent — refreshing rates doesn't affect the balance cache. This prevents unnecessary reloads and keeps the app responsive."

PART 7: THE CONTENTION PATTERN (Reddit Confirmed)

The Reddit candidate said: "*Pattern was contention and scaling reads/writes with a financial twist*."

This is about what happens when multiple things happen simultaneously.

Problem 1: Rate Race Condition

Two users get a rate quote at the same time. The rate changes between their quotes and confirmations.

> "The solution is quote-level locking. Each quote has a UUID and an expiry timestamp. The server validates the specific quote ID at confirmation time — not the 'current rate.' If the quote has expired, the server rejects it and the mobile shows 'Rate expired, fetching new quote.' The user never sees a different rate than what they confirmed."

Problem 2: Balance Race Condition

User sends money from phone AND tablet simultaneously. Only enough balance for one transfer.

> "This is a backend concern — the server uses optimistic locking or serialized writes on the balance. The mobile's role is: generate unique idempotency keys per device, show pending state immediately after confirmation, refresh balance aggressively after writes, and handle 409 Conflict responses gracefully — 'This transfer may have already been processed.'"

Problem 3: Concurrent Token Refresh

Multiple API calls fail with 401 simultaneously. All try to refresh the token.

> "This is exactly what I solved in Round 2. I use a serial queue with a flag — the first caller triggers the refresh, subsequent callers wait for the same result. Only one refresh call hits the server. This prevents token endpoint overload and token revocation from multiple simultaneous refreshes."

What to SAY:

> "The mobile client's role in handling contention is: generating idempotency keys so retries are safe, showing accurate pending states so users don't double-tap, refreshing data aggressively after writes to show the latest state, and gracefully handling conflict responses from the API. The heavy lifting for consistency is on the backend — the mobile's job is to prevent duplicate actions and show honest state."

PART 8: WISE-SPECIFIC TECH TO MENTION

Dynamic Forms

> "Wise built a server-driven UI framework where the backend defines form structure — field types, validation rules, options — and the mobile renders it natively. This is critical for a fintech operating in 40+ countries because regulatory requirements change per country. The backend can add a new KYC field in Lithuania without an app update. I've worked with similar patterns at PayPal where server-driven config controls feature rollouts by country."

Wise's Actual Stack

> "Native iOS with Swift, Android with Kotlin. Java microservices on the backend. Kafka for event-driven messaging. PostgreSQL with read replicas for the database. Redis for caching. Spinnaker for CI/CD. Grafana for monitoring. VIPER architecture on iOS moving toward TCA for new features."

PART 9: TRADE-OFF CHEAT SHEET

For every trade-off, use this template:

> "I'd choose [A] over [B] because [reason]. The trade-off is [what you lose], but in Wise's context [why that's acceptable]. If [requirement changes], I'd reconsider."

Decision	Choose A	Choose B	Why A	Why B
Data fetching	Polling	WebSocket	Most screens, simpler	Rate ticker, real-time critical
Caching	In-memory (NSCache)	Disk (CoreData)	Small volatile data (rates)	Large persistent data (transactions)
Architecture	VIPER	TCA	Existing codebase, team familiarity	New features, SwiftUI-first
Navigation	Router (VIPER)	Coordinator	Single module flow	Multi-step flows (transfer, card order)
Networking	URLSession	Alamofire	Fintech — fewer deps to audit	Non-critical, convenience matters
State	Per-module (VIPER)	Global (TCA)	Feature isolation	Shared state across screens
Storage	UserDefaults	Keychain	Non-sensitive settings	Tokens, card details, financial data
UI updates	Optimistic UI	Wait for server	Balance display (show cached)	Transfer confirmation (must be real)
Image loading	AsyncImage	Kingfisher	Few images	Image-heavy screens (merchant logos)
Feature rollout	Feature flag	Full release	New/risky features	Bug fixes, improvements

---

PART 10: QUESTIONS TO ASK THEM

Pick 2:

1. "How does the mobile team handle Dynamic Forms for the Spend product? Is the card ordering flow server-driven?"
2. "What's the biggest mobile architecture challenge in the Spend product right now?"
3. "How do you manage the VIPER to TCA migration? Screen-by-screen, or is there a broader strategy?"
4. "What's the latency target for card transaction notifications — from card tap to push delivery?"
5. "How do iOS and Android teams coordinate on feature parity?"

READINESS CHECKLIST

- □ Can I start from the CUSTOMER JOURNEY (not the database)?
- □ Can I draw high-level architecture in 2 min?
- □ Can I draw VIPER module and explain each component in 3 min?
- □ Can I walk through transfer flow with debounce + idempotency?
- □ Can I explain cache-first strategy with TTL?
- □ Can I discuss Poll vs WebSocket with trade-offs?
- □ Can I explain event-driven architecture with data buckets?
- □ Can I handle the contention/race condition question?
- □ Can I connect every answer to BFX/PayPal experience?
- □ Can I handle "now add X" by creating new VIPER modules?
- □ Do I mention: testing, feature flags, monitoring, security?
- □ Do I have 3 experience stories ready (2 min each)?
- □ Do I have 2 questions to ask them?